

Model constants (fixed packet input)

Constant	Value	Meaning
CAPACITY	6000	charge units
CC_CURRENT	50	constant-current phase current
CV_VOLTAGE_THRESHOLD	4150	mV; CC->CV transition point
CV_DECAY_STEP	1.2	per tick, CV regulation current
TAPER_CURRENT_THRESHOLD	8	CV->TAPER transition point
TAPER_CURRENT	3	fixed TAPER-mode current
IR_DROP_PER_UNIT	2.0	mV per unit delivered current
BASE_VOLTAGE	3700	mV, at soc=0
VOLT_PER_SOC_PCT	6.0	mV per 1% state of charge
AMBIENT	250	0.1 degC units, starting/ambient temp
HEAT_FULL	6	0.1 degC/tick, full-rate charging
HEAT_DERATED	3	0.1 degC/tick, derated charging
HEAT_TAPER	1	0.1 degC/tick, TAPER mode (always)
COOL_PER_TICK	2	0.1 degC/tick, passive cooling
DERATE_TEMP	450	0.1 degC; derating engages at/above
DERATE_FACTOR	0.5	multiplier on mode current
FAULT_TEMP_HIGH	575	0.1 degC; fault latches at/above
FAULT_TEMP_LOW	520	0.1 degC; release eligible at/below
FAULT_RELEASE_TICKS	4	consecutive cool ticks to release
OVERCURRENT_MAX	45	hard clamp, applied after derating

Figure 1 -- Model constants.

Per-tick update rules, in order (applied every tick, $t=1..320$)

1. Mode transition: CC->CV if PREVIOUS tick's voltage $\geq$ CV_VOLTAGE_THRESHOLD.
CV->TAPER if CV regulation current $\leq$ TAPER_CURRENT_THRESHOLD. One-way only.
2. Mode-commanded current: CC_CURRENT / CV regulation current / TAPER_CURRENT.
3. CV taper-down: in CV mode only, after step 2, reduce CV regulation current by CV_DECAY_STEP (floor 0), for next tick's step 1/2 check.
4. Thermal derating and fault gating: if fault latched, current=0.
Else if INCOMING temperature $\geq$ DERATE_TEMP, current $\ast$ DERATE_FACTOR (applies in every mode, TAPER included). Else current unmodified.
5. Overcurrent clamp: cap current at OVERCURRENT_MAX. Applied AFTER derating (step 4), never before.
6. Heat generation: 0 if current is 0. Else, in TAPER mode: ALWAYS HEAT_TAPER, regardless of temperature -- this packet's own exception, not the derated-heat rule other modes follow when hot.
Else if incoming temp $\geq$ DERATE_TEMP: HEAT_DERATED. Else: HEAT_FULL.
7. Temperature update: temp = incoming temp + heat - COOL_PER_TICK.
8. Fault-latch hysteresis: latch if not already latched and NEW temp $\geq$ FAULT_TEMP_HIGH. If latched: release only after FAULT_RELEASE_TICKS CONSECUTIVE ticks at/below FAULT_TEMP_LOW -- a single tick back above FAULT_TEMP_LOW resets the counter to 0.
9. State of charge: soc = min(100, soc_prev + current / CAPACITY * 100).
10. Terminal voltage: voltage = BASE_VOLTAGE + soc*VOLT_PER_SOC_PCT - current*IR_DROP_PER_UNIT. This tick's own voltage (step 1 of the NEXT tick reads it as the previous tick's voltage).

Figure 2 -- Per-tick update rules, in order.

Figure 3 worked example (fake data)

Certificate format (S03), worked example with FAKE numbers
(these do not correspond to any real tick of this model):

```
CELLGUARD10-CERT-V1
t=1;mode=CC;soc=12.34;temp=317;current=50.00;voltage=3850.50;fault=False
t=2;mode=CC;soc=12.67;temp=321;current=50.00;voltage=3852.50;fault=False
...
t=320;mode=TAPER;soc=77.10;temp=288;current=3.00;voltage=4150.90;fault=False
```

Rules: one line per tick, t=1 through t=320, in order. soc/current/voltage are ALWAYS shown with exactly two digits after the decimal point -- 50.00, never 50 or 50.0 -- even when the value is a whole number or already has fewer significant decimal digits. temp is an integer (0.1 degC units). fault is True or False (or a clearly equivalent boolean rendering). Certificate hash: SHA-256 of the full text (header through the t=320 line, newline-joined, trailing newline included), first 16 hex characters.

Figure 3 -- Certificate format, worked example (fake data).

1. OVERVIEW

A single battery pack is simulated across 320 discrete ticks under a charge and thermal controller. Every tick's state (charge mode, state of charge, temperature, delivered current, terminal voltage, fault latch status) is computed from the immediately preceding tick's state. Figure 1 lists every fixed model constant. Figure 2 lists the ten per-tick update rules, in their required order. Initial state, before tick 1: state of charge 0, temperature AMBIENT, mode CC, fault not latched, hysteresis counter 0, reference voltage BASE_VOLTAGE.

2. CHARGE MODE SEQUENCE

The charge mode is CC, then CV, then TAPER, strictly one-way (never reverts). The CC to CV transition is evaluated against the PREVIOUS tick's reported voltage, never the current tick's own just-computed voltage; on the tick that transition fires, the regulation current is available immediately, that same tick, set to CC_CURRENT -- it is not deferred to the next tick. Once in CV mode, a regulation current value is tracked separately from the delivered current and decays by CV_DECAY_STEP every tick (floored at zero, never negative). The CV to TAPER transition check, and that tick's own mode-commanded current, both read the regulation current as it enters the tick -- before that same tick's own decay step runs, the same "previous value" convention the voltage check above already uses. In TAPER mode, delivered current (before derating/clamping) is always the fixed TAPER_CURRENT. The CV regulation-current decay is unconditional for every CV-mode tick -- it is not gated by whether a fault is latched, which only zeroes the delivered current (section 5), never the regulation current. The CV to TAPER transition can therefore fire, and does in this model, on a tick where the fault is still latched and delivered current is zero -- do not assume mode progression pauses during a fault.

3. THERMAL DERATING AND THE OVERCURRENT CLAMP

If the pack's incoming temperature for a tick (its temperature before that tick's own heat/cool update) is at or above DERATE_TEMP, the mode-commanded current for that tick is multiplied by DERATE_FACTOR -- in every mode, TAPER included. The clamp step itself always executes, every tick, in every mode, AFTER derating, never before -- but it only changes the delivered value (binds) when the post-derate current still exceeds OVERCURRENT_MAX. In this model's CC phase, CC_CURRENT (50) exceeds OVERCURRENT_MAX (45), so the clamp binds and reduces delivered current to 45 for as long as derating has not yet engaged; once temperature reaches DERATE_TEMP partway through the CC phase, derating already brings current down to 25 before the clamp step runs, so the clamp executes but is a no-op for the remainder of the CC phase. Do not assume the clamp binds uniformly across the whole CC phase -- trace your own engine's incoming-temperature values to find exactly where derating engages.

4. HEAT GENERATION

If delivered current is zero (fault latched), no heat is generated that tick. TAPER mode is an exception to the derating-sensitive heat rule that every other mode follows: a TAPER-mode tick always generates HEAT_TAPER, regardless of whether incoming temperature is at or above DERATE_TEMP. Only CC and CV mode ticks follow the general rule: incoming temperature at or above DERATE_TEMP generates HEAT_DERATED; otherwise HEAT_FULL. Temperature is then updated by adding that tick's heat and subtracting COOL_PER_TICK.

5. FAULT LATCH

If the pack is not currently fault-latched and the NEW temperature (after this tick's own update) is at or above `FAULT_TEMP_HIGH`, the fault latches immediately and the release-hysteresis counter resets to zero. While latched, the pack delivers zero current every tick (regardless of mode) until released. Release requires the temperature to be at or below `FAULT_TEMP_LOW` for `FAULT_RELEASE_TICKS` CONSECUTIVE ticks -- the counter increments only while temperature stays at or below `FAULT_TEMP_LOW` every tick in a row; a single tick where temperature rises back above `FAULT_TEMP_LOW` resets the counter to zero immediately, even if the pack had been close to releasing. The instant the counter reaches `FAULT_RELEASE_TICKS` and the fault un-latches, the counter itself resets to zero in that same tick.

6. STATE OF CHARGE AND VOLTAGE

State of charge increases each tick by delivered current divided by `CAPACITY`, times 100, capped at 100. Terminal voltage is `BASE_VOLTAGE` plus state of charge times `VOLT_PER_SOC_PCT`, minus delivered current times `IR_DROP_PER_UNIT` -- this tick's own voltage, which becomes "the previous tick's voltage" for the NEXT tick's mode-transition check (section 2), never used for this same tick's own transition decision. Every state variable (state of charge, temperature, the CV regulation current, voltage) carries full, unrounded numeric precision from tick to tick; the certificate's 2-decimal-place rounding (Figure 3) is a display rule applied once at serialization, and never feeds back into the next tick's computation.

7. INDEPENDENT VERIFICATION

Deliver a separately coded verifier that re-implements these ten rules itself, from scratch, in its own code -- never importing, calling, or reading any state computed by the primary engine. It must check the primary's claimed state at several checkpoint ticks by independently replaying the simulation from tick 1 up to each checkpoint using only its own logic. Run two required adversarial mutations against it: a claimed fault release after only two consecutive cool ticks instead of four, and a claimed delivered current above `OVERCURRENT_MAX` as if the clamp were never applied. The verifier must reject both while still accepting the true trace's own correct checkpoint states.

8. CERTIFICATION

See Figure 3 for the exact, fully mechanical certificate serialization format and a worked example using fake numbers. Deliver six products as files: the engine source; the independent verifier source; the full 320-tick state trace; a findings report; certification evidence (both adversarial rejection results plus the SHA-256 trace-integrity hash); and an engineering memo explaining, citing your own executed engine's actual reported values: why TAPER mode never follows the derated-heat rule even while hot; why the fault latch released at the specific tick it did, citing the consecutive-tick hysteresis counter; and exactly which CC-phase ticks the overcurrent clamp actually changes the delivered current on, and why it stops changing it partway through. Base every reported value on your own executed implementation -- never hand-derive or embed a precomputed state, checkpoint value, or hash as a substitute for running the delivered engine.